



**Arab Academy for Science and Technology Maritime
Transport**

**College of Computing and Information Technology
Computer Science**

Computer Graphics

Presented to :

Dr/ Hassan El-Ansary

Eng/ Youssef Alaa

Presented by :

Marslino Wael 231008561

Remon Ayman 231018419

Beshoy Kameal 231008629

Mario Kamal 241009379

Ahmed Mohamed 231008997

Mostafa Ayman 231018299

1. Introduction

This project demonstrates the implementation of core 2D computer graphics algorithms from scratch using C++ and OpenGL (FreeGLUT).

Instead of relying on built-in high-level graphics library functions, all rasterization, mathematical transformations, and clipping operations were implemented manually to provide a deep understanding of the mathematical and geometric foundations of computer graphics.

The project also includes a modern interactive graphical user interface (GUI), real-time performance metrics, an Undo/Redo system, and scene serialization for saving and loading work.

2. Objectives

- Implement and analyze line drawing algorithms:
 - DDA (Digital Differential Analyzer)
 - Bresenham Line Algorithm
- Implement circle drawing algorithms:
 - Midpoint Circle Algorithm
 - Bresenham Circle Algorithm
- Apply matrix-based 2D transformations:
 - Translation
 - Rotation
 - Scaling
 - Reflection
 - Shearing
- Implement clipping algorithms:
 - Cohen-Sutherland Line Clipping
 - Sutherland-Hodgman Polygon Clipping
- Generate parametric curves:
 - Cubic Bezier Curves

- B-Spline Curves
 - Compare performance, efficiency, and accuracy of all implemented algorithms.
-

3. Algorithm Descriptions

3.1 Line Drawing Algorithms

DDA (Digital Differential Analyzer)

The DDA algorithm uses floating-point increments based on the line equation:

$$y = mx + c$$

It calculates step values for x and y and rounds each computed coordinate to the nearest pixel.

Advantages:

- Simple to understand and implement.

Disadvantages:

- Slower due to floating-point calculations and rounding operations.
-

Bresenham Line Algorithm

Bresenham uses an integer decision parameter to determine the next pixel position.

Advantages:

- Avoids floating-point operations.
 - Faster and more efficient for raster displays.
 - Uses only integer addition and subtraction.
-

3.2 Circle Drawing Algorithms

Midpoint Circle Algorithm

This algorithm uses the decision function:

$$f(x, y) = x^2 + y^2 - r^2$$

It determines whether the next pixel should move East or South-East.

Features:

- Calculates only one octant of the circle.
- Uses 8-way symmetry to mirror points and complete the full circle.

Advantages:

- Fast and highly efficient.
-

Bresenham Circle Algorithm

This algorithm minimizes the error between the selected pixel and the actual circle boundary.

Advantages:

- Stable integer calculations.
 - High drawing accuracy.
 - Efficient execution.
-

3.3 2D Transformations

All transformations are implemented mathematically relative to a pivot point.

Translation

Moves an object by adding displacement values:

(dx, dy)

to all vertices.

Rotation

Uses sine and cosine functions to rotate shapes around a pivot point.

Scaling

Resizes objects uniformly or non-uniformly.

Shearing

Slants shapes horizontally or vertically.

Reflection

Reflects objects across:

- X-axis
 - Y-axis
 - Origin
-

3.4 Curves and Clipping

Bezier Curves

Cubic Bezier curves are generated using Bernstein polynomials.

Features:

- Smooth curve generation.
 - Global control over curve shape.
-

B-Spline Curves

B-Spline curves use basis functions.

Advantages:

- Local control of shape.
- Modifying one control point does not affect the entire curve.

Disadvantages:

- More computationally intensive.
-

Cohen-Sutherland Line Clipping

Uses 4-bit outcodes for each point.

Advantages:

- Fast accept/reject operations using bitwise logic.
-

Sutherland-Hodgman Polygon Clipping

Clips polygons against all four clipping window boundaries sequentially.

Supports:

- Convex polygons
 - Complex polygon clipping operations
-

4. Performance Analysis

Algorithm	Performance Notes	
DDA Line	Slower	Floating-point operations and rounding overhead
Bresenham Line	Very Fast	Integer-only calculations
Midpoint Circle	Fast	Uses 8-way symmetry
Bezier Curve	Moderate	Iterative calculations over parameter t
B-Spline Curve	Heavy	Basis function computations
Cohen-Sutherland Fast		Bitwise logical operations

5. Challenges Faced

- Managing OpenGL and FreeGLUT dependencies.
 - Integrating CMake correctly across different systems.
 - Synchronizing screen coordinates with OpenGL viewport coordinates.
 - Handling floating-point precision in curve generation.
 - Implementing polygon clipping edge cases correctly.
 - Building Undo/Redo state management.
 - Implementing Scene Serialization for Save/Load functionality.
-

6. Screenshots

Insert project screenshots in the following sections:

1. Main Interface
2. Transformations Demonstration
3. Clipping Demonstration
4. Split-Screen Comparison

(Replace this section with actual screenshots before submission.)

7. Conclusion

This project successfully implements all required computer graphics algorithms while providing an interactive environment for experimentation and visualization.

The comparison mode demonstrates the practical efficiency advantages of integer-based algorithms such as Bresenham over floating-point algorithms like DDA.

Additional features such as:

- B-Spline Curves
- Undo/Redo System
- Scene Serialization

extend the project beyond the minimum requirements and transform it into a small graphics engine suitable for educational and experimental purposes.

GitHub: <https://github.com/RemonAyman/computergraphics>

Presentation link: <https://canva.link/gcf2eabkub6o2qp>